CLAUDE CODE DEEP DIVE WORKSHOP · FSI EDITION

Chapter 6 - 토큰과 비용

덜 쓰고 , 보고 , 막기

Update : 2026.11 / Claude Code v2.1 최신 채널 · Amazon Bedrock 기준 , 공식 문서 code.claude.com/docs 반영

Choi, WooHyung
Prin. Solutions Architect
AWS Korea
whchoi@amazon.com

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

Chapter 6 Agenda

2 Parts / 40 분 + 50 분 · 심화 +20 분

PART 01	컨텍스트 엔지니어링	7 가지 원리 , 공식 기능 + 커뮤니티 사례	13:00 · 40 분
PART 02	조직의 비용 통제	OTel · modelPricing · Gateway	16:50 · 50 분
DEEP DIVE	심화 (선택)	Part 별 10 분씩 · 스피커 재량	+20 분

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

06

토큰과 비용

컨텍스트 엔지니어링 · 계측 · 조직 통제

CHAPTER 06 / PART 01

컨텍스트 엔지니어링

같은 일을 더 적은 토큰으로

1. 보이는 것부터
2. 상주 지침 줄이기
3. 구조 정보 , 출력 필터 , 도구 지연 로드
4. 캐시와 압축 (compaction)
5. 모델과 사고량

BASELINE

$13 / 활성일

엔터프라이즈 배포에서 개발자 1인당 평균 . 월 $150–250, 90% 는 활성일 $30 미만

출처 : code.claude.com/docs/en/costs · 우리 팀의 기준선은 파일럿에서 직접 측정

원리 1 · 먼저 본다

/context 와 /usage 로 무엇이 자리를 차지하는지

/usage · Session

```
Total cost:            $0.55
Total duration (API):  6m 20s
Usage by model:
  claude-sonnet-4-6:  1.2k input, 5.3k output,
                      940.0k cache read, 50.0k cache write
Prompt cache (main):   14 requests · 91% of input tokens
                       from cache · 2 misses · warm (1h TTL)
```

읽는 법

- /context: 항목별 컨텍스트 점유
- /usage: 세션 비용 · 캐시 적중률
- 캐시 miss 에는 추정 원인이 표시됨
- 금액은 추정치 , 정산은 AWS 청구서

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

여덟 가지 원리

본론은 공식 기능 , 커뮤니티 도구는 사례로만

#	원리	공식 기능	커뮤니티 사례
1	먼저 본다	/context, /usage, 스피너 토큰 수	—
2	상주 지침은 짧게	CLAUDE.md 200 줄 , skills, SessionStart hook	Ponytail
3	탐색 대신 구조 정보	code intelligence 플러그인 , 개요 skill	Graphify
4	큰 출력은 들어오기 전에	PreToolUse 필터 hook, subagent 격리	Headroom
5	도구는 필요할 때만	MCP 지연 로드 , 서버 끄기 , CLI 우선	REA
6	캐시를 깨지 않는다	prompt caching, TTL, /clear vs /compact	—
7	압축은 내가 정한 때에	/compact 지시 , autoCompactWindow, idle compaction	—
8	작업에 맞는 모델 · 사고량	/model, /effort, subagent model	—

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

원리 2 · 상주 지침은 짧게

매 요청에 실리는 것과 필요할 때 오는 것

CLAUDE.md (상주)

- 세션 시작마다 로드 , 매 요청에 포함
- 목표 : 200 줄 이하 , 필수만
- PR 리뷰 ·DB 마이그레이션 절차는 여기에 두지 않음

Skills (요청 시)

- 평소에는 설명만 상주
- 호출될 때 본문 로드
- 작업별 절차는 skill 로 이동
- SessionStart hook 으로 짧은 규칙만 주입

커뮤니티 사례 · 비공식 · 효과 수치는 제작자 자체 보고

Ponytail · SessionStart hook 으로 약 100 줄의 YAGNI 규칙을 주입하는 플러그인 . 독립 테스트에서는 비용 약 10% 감소

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

원리 3 · 탐색 대신 구조 정보

grep 과 파일 여러 개 읽기를 한 번의 질의로

OFFICIAL

Code intelligence 플러그인

언어 서버로 정의로 이동
편집 후 타입 오류 자동 보고
불필요한 파일 읽기 감소

OFFICIAL

코드베이스 개요 skill

아키텍처 · 디렉터리 · 명명 규칙을 skill 로
탐색에 쓰는 토큰을 줄임

커뮤니티 사례 · 비공식 · 효과 수치는 제작자 자체 보고

Graphify · 코드를 로컬 AST 로 파싱해 지식 그래프로 만들고 skill·MCP 로 제공 . 작은 저장소에서는 효과가 거의 없음

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

원리 4 · 큰 출력은 들어오기 전에

공식 예제 : 테스트 출력을 실패 줄만 남기기

~/.claude/hooks/filter-test-output.sh

```
#!/bin/bash
input=$(cat)
cmd=$(echo "$input" | jq -r '.tool_input.command')

pattern='^(npm test|pytest|go test)( [^;&|<>`$#\\[:cntrl:]]*)?$'
if [[ "$cmd" =~ $pattern ]]; then
  filtered_cmd="$cmd 2>&1 | grep -A 5 -E '(FAIL|ERROR|error:)' | head -100"
  echo "$input" | jq --arg filtered "$filtered_cmd" \
    '{hookSpecificOutput: {hookEventName: "PreToolUse",
      updatedInput: (.tool_input + {command: $filtered})}}'
else
  echo "{}"
fi
```

효과

- PreToolUse 가 명령을 다시 씀
- 수만 토큰 로그 → 수백 토큰
- 장황한 작업은 subagent 로 격리
- settings.json PreToolUse · matcher Bash

커뮤니티 사례 · 비공식 · 효과 수치는 제작자 자체 보고

Headroom · 요청 경로에 프록시를 두고 도구 출력 · 로그를 압축 . ⚠ 프록시가 모든 프롬프트를 지나가게 됨

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

DEMO · 7 분

출력 필터 훅 적용 전후 비교

1. 실패 테스트가 섞인 저장소에서 "npm test 돌려서 실패 원인 찾아줘 "
2. /context 와 /usage 기록
3. filter-test-output.sh 등록 후 /clear, 같은 요청 반복
4. 두 번의 컨텍스트 크기와 비용 비교

강사 데모 · 참가자는 Lab 포털의 같은 스크립트로 따라 해 볼 수 있습니다

원리 5 · 도구는 필요할 때만

MCP 서버는 공짜가 아닙니다

DEFERRED

도구 정의 지연 로드

기본값 : 도구 이름과 서버 지침만
쓸 때 스키마를 tool search 로 로드
⚠ 커스텀 BASE_URL 게이트웨이는 확인

TRIM

안 쓰는 서버 끄기

/mcp 로 서버 목록 확인
쓰지 않는 서버는 비활성화

CLI FIRST

CLI 우선

gh, aws 같은 CLI 는
도구 목록을 상주시키지 않음

커뮤니티 사례 · 비공식 · 효과 수치는 제작자 자체 보고

REA (Reverse Engineer Anything) · 170 여 개 MCP 도구를 가진 역공학 도구 . 도구 목록이 거대해 지연 로드가 왜 기본값인지 보여 주는 사례 . 비신뢰 바이너리는 VM· 법무 검토

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

QUICK CHECK · 손을 들어 답해 주세요

Q. 1 시간 넘게 자리를 비운 뒤 보내는 첫 메시지는 ?

A

캐시에서 읽어 저렴하다

B

캐시 미스로 전체 컨텍스트를 다시 처리한다

C

자동으로 /clear 된다

정답 B · 기본 캐시 수명은 5 분 . 긴 세션일수록 비용이 큼

원리 6 · 캐시를 깨지 않는다

/clear 와 /compact 는 비용이 다릅니다

/clear

- 새 세션 시작 , 비용 0
- 관련 없는 작업으로 넘어갈 때
- /rename 후 /resume 으로 돌아오기

/compact

- 대화를 읽어 요약 = 그 자체가 큰 요청
- 이어서 작업해야 할 때만
- 보존할 것을 지시 : /compact Focus on ...
- CLAUDE.md 에 압축 지침을 둘 수 있음

5 분
기본 캐시 수명

1 시간
1H 옵션 , 더 비쌈

miss 원인
/usage 에 표시

Bedrock
모델 · 리전별 지원 확인

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

원리 7 · 압축은 내가 정한 때에

대화가 차면 Claude Code 는 오래된 기록을 요약합니다

방식	언제	다이얼
자동 압축	auto-compact window 에 닿을 때	1M 모델 기본 약 967K · /autocompact 400k · autoCompactWindow
/compact [지시]	내가 원할 때	/compact focus on … · CLAUDE.md # Compact instructions
/rewind 부분 요약	특정 메시지부터 · 까지	Summarize from here / up to here
Idle compaction	1 시간 TTL 세션을 오래 비웠을 때 , 캐시 만료 직전	v2.1.290+ · idleCompaction: false 로 끔
/clear	관련 없는 작업으로 전환	비용 0, 연속성 없음

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

auto-compact window 조정

다 차기 전에 미리 압축해 요청당 입력을 줄인다

Claude Code 세션 · Terminal

```
# 현재 값과 출처 확인
/autocompact

# 400K 에서 압축 (모델별로 사용자 설정에 저장, v2.1.288+)
/autocompact 400k

# 모델 기본값으로 복귀
/autocompact auto

# 이번 세션만
claude --autocompact 500k

# 조직 기본값 (managed-settings.json)
{ "autoCompactWindow": 400000 }
```

우선순위

- ① 환경 변수 (최우선)
- CLAUDE_CODE_AUTO_COMPACT_
- WINDOW
- ② --autocompact 실행 플래그
- ③ settings: managed 가 사용자 /autocompact 를 덮음
- 같은 파일에서는 modelSettings.<model> 값이 우선
- 범위 100K~1M 또는 "auto", 실제 시점은 모델 최대 컨텍스트와 중 작은 쪽

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

창을 줄이면 항상 싸질까

Claude Code 도 기본값 "auto" 를 권합니다

줄어드는 것

- 압축 전까지 매 요청의 입력 토큰
- 캐시가 식은 뒤 다시 읽는 양
- 1M 모델의 긴 세션에서 효과가 큼

늘어나는 것 · 잃는 것

- 압축 횟수 : 한 번이 대화 전체를 읽는 요청
- 압축 직후 캐시 재구축
- 손실 요약 : 초반 제약 · 정확한 오류가 빠질 수 있음
- 기본값을 바꾸면 긴 세션 재개 시 사용량이 늘 수 있다는 공식 경고

1 순위
작업 단위 끝에 캐시가 따뜻할 때 /compact

2 순위
긴 세션 팀만 창을 400K~500K 로

검증
OTel token.usage 로 전후 비교

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

압축 후에 무엇이 남나

공식 문서 : What survives compaction

항목	압축 후
시스템 프롬프트 , 출력 스타일	그대로 적용
루트 CLAUDE.md, auto memory, plan 모드의 계획	디스크에서 다시 주입
최근 읽거나 고친 파일	최근 수정 순으로 최대 5 개 다시 읽음 (5,000 토큰 넘으면 경로만)
사용한 스킬 본문	다시 주입 , 스킬당 5,000 · 합계 25,000 토큰 , 오래된 것부터 제외
paths: 규칙 , 하위 폴더 CLAUDE.md	요약되어 사라짐 , 해당 파일을 다시 읽을 때 재로드
SessionStart 훅 (matcher: compact)	다시 실행되어 결과가 추가됨

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

압축의 비용 · 캐시가 따뜻할 때와 식었을 때

/compact 도 대화 전체를 읽는 요청입니다

캐시가 따뜻할 때

- 요약 요청이 접두부를 캐시에서 읽음
- 비용은 컨텍스트 크기의 일부
- 작업 사이에 직접 /compact
- /rewind·/recap 은 캐시를 그대로 둠

TTL 이 지난 뒤

- 전체 기록을 비캐시 입력으로 다시 처리
- 오래된 세션을 재개한 직후의 /compact 가 가장 비쌈
- Bedrock 기본 TTL 은 5 분
- 그래서 자리 비우기 전에 /compact 또는 /clear

/usage
expected rebuild 로 따로 집계

추천
Compact before you step away

/rewind
이미 캐시된 지점으로 복귀

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

Bedrock 에서 캐시 수명과 idle compaction

같은 기능도 접속 방식에 따라 동작이 다릅니다

접속 방식	메인 대화 기본 TTL	Idle compaction
Claude 구독 (플랜 사용량 안)	1 시간	동작
API 키 · Amazon Bedrock	5 분	기본으로는 해당 없음
Bedrock + promptCacheTtl: "1h"	1 시간 (모델별 지원 확인)	feature flag 조회가 필요해 Bedrock 에서는 동작하지 않는다는 보고 · 리허설 확인
Claude apps gateway 경유	1 시간 TTL 미지원	해당 없음

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

조직 기본값으로 압축 설계

managed settings 로 창 크기 , 캐시 수명 , 압축 후 규칙을 고정

managed-settings.json

```
{
  "autoCompactWindow": 400000,
  "promptCacheTtl": "1h",
  "hooks": {
    "SessionStart": [
      { "matcher": "compact",
        "hooks": [{ "type": "command",
          "command": "cat /etc/claude-code/dlp-reminder.md" }] }
    ],
    "PreCompact": [
      { "hooks": [{ "type": "command",
          "command": "/opt/audit/save-transcript.sh" }] }
    ]
  }
}
```

설계 포인트

- autoCompactWindow: 100K~1M, 일찍 압축하면 요청당 입력이 작아짐
- promptCacheTtl 1h 는 자리 비움이 잦은 팀에만
- idle compaction 은 Bedrock 에서 기대하지 말 것 → 떠나기 전 /compact
- 압축 뒤 DLP 규칙은 SessionStart compact 로 재주입
- PreCompact: exit 2 로 압축 차단 , 감사용 보관
- Haiku 5.5 는 100K 넘는 프롬프트 단가가 더 높음

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

QUICK CHECK · 손을 들어 답해 주세요

Q. Bedrock 세션을 1시간 비운 뒤 돌아와 /compact를 치면?

A

캐시에서 읽어 저렴하다

B

전체 기록을 비캐시로 다시 읽는 큰 요청이 된다

C

idle compaction이 이미 처리했다

정답 B · Bedrock 기본 TTL 5분, 자리 비우기 전에 /compact 또는 /clear

LAB · 15 분

압축을 내가 정한 때에

1. /context · /usage 로 압축 전 기록
2. /compact 테스트 실패 원인과 남은 할 일만 남겨줘
3. CLAUDE.md 에 # Compact instructions 추가
4. SessionStart compact 훅으로 사내 규정 재주입 확인
5. /autocompact 400k 로 창 조정 → 저장 위치 확인 → auto 로 복귀
6. /config 에서 Idle compaction 항목과 Bedrock TTL 확인

Hands-on Lab 포털 → Chapter 6 TASK 04

원리 8 · 작업에 맞는 모델과 사고량

사고 토큰은 출력 단가로 청구됩니다

MODEL

/model

대부분의 코딩은 Sonnet
복잡한 설계만 Opus

EFFORT

/effort

단순 작업은 effort 를 낮춤
5.5 세대와 Fable 은 thinking 을 끌 수 없음

SUBAGENT

model: haiku

단순 subagent 는 작은 모델
고정 예산 모델은 MAX_THINKING_TOKENS

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

커뮤니티 도구를 볼 때의 세 가지 질문

도입 전 5 분 검토

- **어느 원리를 쓰는가**
  여덟 가지 원리 중 무엇이고 , 어느 확장 지점 (hook·skill·MCP· 프록시) 에 붙는가

- **공식 기능으로 되는가**
  CLAUDE.md, skill, 필터 hook, code intelligence 로 같은 효과를 낼 수 있는가

- **데이터는 어디로 가는가**
  로컬 처리인지 , 제 3 자로 가는지 . 도입한다면 사내 마켓플레이스에 고정 버전으로 (Ch3·Ch5)

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

CHAPTER 06 / PART 01 DEEP DIVE

심화 · 스피커 재량

약 10 분 · 골라 쓰세요

1. MCP 도구 정의 비용
2. 세션 위생
3. 모델 배분 예시

MCP 도구 정의 비용

연결만 해도 컨텍스트를 씁니다

상황	영향	대응
MCP 서버 다수 연결	도구 스키마가 매 요청에 실림	/mcp 에서 쓰지 않는 서버 끄기
도구가 많은 서버	정의만으로 수천 토큰	도구 지연 로드 (필요 시 검색)
같은 기능 중복	선택 혼란 , 재시도 증가	서버 하나로 정리
프로젝트별 차이	불필요한 서버 상주	.mcp.json 을 프로젝트 단위로

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

세션 위생

같은 일을 더 적은 토큰으로

CLEAR

/clear

작업이 바뀌면 새 컨텍스트
이전 대화는 비용만 남김

COMPACT

/compact 지시

남길 것을 지정해 요약
예 : 결정 사항과 남은 할 일만

CACHE

자리 비운 뒤

5 분 넘으면 캐시 미스
긴 휴식 후엔 /clear 고려

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

모델 배분 예시

역할별로 모델을 고정

.claude/agents/log-summarizer.md

```
---
name: log-summarizer
description: 긴 테스트 · 빌드 로그를 실패 원인 중심으로 5 줄 요약한다 .
로그 분석 요청 시 사용 .
tools: Read, Grep
model: haiku
---
실패한 테스트 이름 , 첫 오류 메시지 , 의심 파일만 보고한다 .
```

효과

- 장황한 로그는 작은 모델이 읽음
- 메인에는 5 줄만 돌아옴
- 메인 모델은 판단에 집중
- Bedrock 계정에 Haiku 활성화 확인

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

CHAPTER 06 / PART 02

조직의 비용 통제

쓴 만큼 보고 , 넘치면 막는다

1. Bedrock 에서 비용을 보는 법
2. OTel 대시보드
3. modelPricing 과 AWS 쪽 도구
4. Claude apps gateway (참가자 실습)

셋업별 비용 관리

Bedrock 은 클라우드 콘솔과 직접 깐 계측이 전부입니다

셋업	지출 보기	상한	사용자별
Claude Team·Enterprise	조직 Analytics	관리 콘솔 spend limit	CSV · Analytics API
Claude Console	Console Usage	Workspace limit	Claude Code Analytics API
Amazon Bedrock	AWS 청구 콘솔	AWS 예산 도구	OTel · Gateway

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

BEDROCK REALITY

Bedrock 에는 Analytics 대시보드가 없습니다

— 계측을 깔아야 보이고 , 안 깐 사람은 잡히지 않습니다

OTel 을 정책으로 켜기

managed settings env 한 블록

managed-settings.json (env)

```
{
  "env": {
    "CLAUDE_CODE_ENABLE_TELEMETRY": "1",
    "OTEL_METRICS_EXPORTER": "otlp",
    "OTEL_LOGS_EXPORTER": "otlp",
    "OTEL_EXPORTER_OTLP_PROTOCOL": "http/protobuf",
    "OTEL_EXPORTER_OTLP_ENDPOINT": "https://otel.corp.example.internal",
    "OTEL_RESOURCE_ATTRIBUTES": "department=platform,team.id=core"
  }
}
```

대표 지표

- claude_code.token.usage
- claude_code.cost.usage
- claude_code.session.count
- lines_of_code · commit · PR
- resource 속성으로 팀 귀속

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

DEMO · 15 분

ccdash.atomai.click · 이틀간 우리가 쓴 토큰

1. 랩 환경은 Day 1 부터 OTel 수집 중 (참가 안내문 고지)
2. 전체 토큰 · 비용 추이 , Capstone 구간 피크 확인
3. 모델별 · 도구별 분포 , 캐시 적중률
4. 내 참가자 ID 로 내 사용량 찾기
5. 가장 비용이 컸던 패턴을 Part 1 원리로 해석

화면에는 참가자 ID 만 표시합니다 · 회사별 사용량은 노출하지 않음

계약 단가로 보여 주기

추정 비용을 청구서에 가깝게

managed-settings.json

```
{
  "modelPricing": {
    "multiplier": 0.9
  }
}

# /usage → Total cost ... at your organization's configured rates
```

포인트

- managed settings 에서만 유효 (v2.1.242+)
- multiplier < 1 할인 , > 1 할증
- 모델별 단가는 overrides 로
- 표시만 바꿈 , 청구는 그대로
- 숫자는 계약 단가를 입력

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

AWS SIDE

AWS 쪽 비용 도구

Claude Code 의 계측과 별개로 , AWS 계정 쪽에서 볼 수 있는 것들입니다 . 둘을 대사해야 정확합니다 .

01 **Application inference profile 태그**

Ch2 의 modelOverrides 로 팀별 경로 → Cost Explorer

02 **AWS Budgets**

계정 · 태그 단위 예산 알람

03 **전용 AWS 계정**

Claude Code 비용을 계정 경계로 분리 (공식 권장)

04 **TPM 쿼터**

라이브 교육 같은 동시 사용 급증은 사전 증액

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

ORG CONTROL

Claude apps gateway

SSO 로그인 · IdP 그룹별 모델과 정책 · 사용자별 spend limit · OTLP 중계 . Bedrock 앞단에 직접 운영하는 게이트웨이입니다

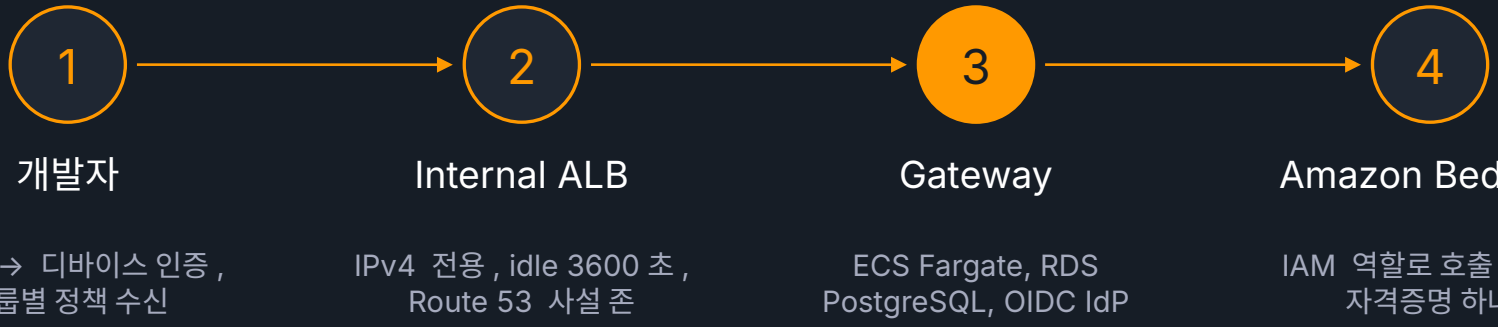
claude 바이너리에 포함 · claude gateway --config gateway.yaml

Gateway 구성 (공식 AWS 예제)

사설망 전용 · internal ALB · IPv4

PRIVATE ONLY

/login 은 게이트웨이 호스트가 사설 IP 로만 resolve 될 때만 통과합니다 . 로그인 브라우저도 사설망 안에 있어야 합니다 .



© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

이번 랩의 Gateway 구성

참가자 계정마다 VPC 안에 한 대씩

LAB TOPOLOGY

CLI 는 사설 존으로 , 로그인 브라우저는 CloudFront → ccb nginx 로 . 게이트웨이는 VPC CIDR 만 허용합니다 .



ccb-endpoint

use_gateway → managed settings 에 forceLoginGatewayUrl

Route 53 사설 존

gateway.workshop.internal · 자체 서명 TLS → trust 확인

Gateway EC2

Docker: claude gateway + PostgreSQL · Cognito OIDC (alice / bob)

Amazon Bedrock

게이트웨이 인스턴스 역할 · us-east-1 global 5.5

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

Spend limits

사용자별 상한을 실시간으로 강제

항목	동작	메모
범위	user (OIDC sub) · rbac_group · organization	그룹 · 조직 상한은 1 인당 기본값
기간	daily · weekly · monthly (UTC 기준 리셋)	기간별로 각각 적용
초과 시	429 billing_error + 리셋 시각	75%·95% 에서 사전 경고
단가	pricing.overrides → 목록가 → 미지 모델 $5/$25	추정치 , 정산은 AWS
설정	Admin API POST /v1/organizations/spend_limits	gateway.yaml 이 아님

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

운영 전에 정할 두 가지

기본값이 금융권 기대와 다를 수 있습니다

저장소 장애 시

- 기본은 fail-open: 요청 통과 , 경고 로그
- enforcement.fail_closed_on_error: true 면 차단
- 추론 중단 vs 미계측 지출 중 선택

개인정보

- principal_emails 테이블에 이메일 · 이름 · 그룹 저장
- 기본 90 일 보존 (identity_retention_days)
- 퇴사 · 열람 요청 시 해당 행 삭제
- spend·audit 은 OIDC sub 만 참조

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

QUICK CHECK · 손을 들어 답해 주세요

Q. standard-users 그룹 한도가 $50/ 일인데 bob 에게만 1 센트 사용자 한도를 걸면 ?

A

그룹 한도 $50 이 우선한다

B

사용자 한도가 우선해 다음 요청부터 429 로 막힌다

C

진행 중인 응답까지 즉시 끊긴다

정답 B · 사용자 한도 > 그룹 기본값 , 진행 중 응답은 끝까지 감

LAB · 5 분

텔레메트리는 어디로 가는가

1. settings.json 의 OTEL_* 환경 변수 읽기
2. otelcol · fanout 타이머 active 확인
3. scrub_logs: 내보내기 전 프롬프트 본문 삭제 규칙 찾기
4. exporters 에서 CloudWatch + 중앙 대시보드 이중 전송 확인

Hands-on Lab 포털 → Chapter 6 TASK 05

LAB · 20 분

Gateway 실습 · 모델 정책과 지출 한도

1. use_gateway → alice 로그인 , /model 에 Opus 포함 · /usage Spend limit
2. bob 으로 재로그인 → Opus 없음 , 질문 1 개로 지출 기록
3. Terminal 2: Admin API 로 bob 조회 → 사용자 한도 1 센트
4. bob 다음 요청이 spend limit 메시지로 거절
5. 한도 DELETE → 그룹 한도 복원 → use_bedrock_direct

각자 계정의 게이트웨이 · Hands-on Lab 포털 → Chapter 6 TASK 06~07 · 게이트웨이 장애 팀은 강사 화면

CHAPTER 06 / PART 02 DEEP DIVE

심화 · 스피커 재량

약 10 분 · 골라 쓰세요

1. OTel 지표 사전
2. 이상 징후 패턴
3. Admin API 로 한도 운영

OTel 지표 사전

대시보드의 숫자는 어디서 오나

지표	속성	용도
claude_code.token.usage	type: input · output · cacheRead · cacheCreation	캐시 효율 , 모델별 사용량
claude_code.cost.usage	model	추정 비용 (정산은 AWS)
claude_code.session.count	user · team 속성	활성 사용자
claude_code.lines_of_code.count	type: added · removed	생산성 지표
claude_code.commit.count · pull_request.count	-	산출물
claude_code.active_time.total	-	실제 사용 시간

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

이상 징후 패턴

숫자를 보고 무엇을 의심하나

패턴	의심	처방
cacheRead 비율이 낮음	긴 휴식 후 재개 , 접두부가 자주 바뀜	세션 위생 , CLAUDE.md 안정화
input 토큰 급증	큰 로그 · 파일을 통째로 읽음	출력 필터 훅 , subagent 격리
Opus 비중이 높음	기본 모델 미고정	ANTHROPIC_DEFAULT_*, availableModels
특정 사용자 급증	자동화 루프 , 병렬 남용	게이트웨이 사용자 한도
분류기 호출 비중 큼	auto mode 분류기가 큰 모델로 폴백	CLAUDE_CODE_AUTO_MODE_MODEL 명시 (Ch3)

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

Admin API 로 한도 운영

그룹 기본값 + 예외 사용자

Terminal

```
# 그룹 기본값 : standard-users 1 인당 하루 $50
curl -X POST $GW/v1/organizations/spend_limits -H "x-api-key: $WRITE" \
  -H "Content-Type: application/json" \
  -d '{"scope":{"type":"rbac_group","rbac_group_id":"standard-
users"},"amount":"5000","period":"daily"}'

# 현재 적용 한도와 사용액
curl "$GW/v1/organizations/spend_limits/effective?period[]=daily" -H "x-
api-key: $READ"
```

운영 규칙

- amount 는 US 센트 문자열
- 그룹 한도는 공유 풀이 아닌 1 인당 기본값
- 사용자 한도가 그룹보다 우선
- write 키는 소수만 , 비밀 저장소에 보관

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

선택 LAB · 10 분

내 계정에서 비용 다이얼 적용

1. managed settings 에 modelPricing.multiplier 병합
2. /usage 에서 configured rates 표시 확인
3. 대시보드에서 적용 전후 내 사용량 비교
4. 백업으로 원복

각자 Bedrock 직결 · Hands-on Lab 포털 → Chapter 6 선택 TASK 08 · 스피커 재량

Chapter 6 Recap

덜 쓰고 , 보고 , 막기

01 먼저 본다 : /context, /usage, 캐시 적중률

02 상주 지침은 짧게 , 큰 출력은 들어오기 전에

03 커뮤니티 도구는 원리로 읽고 공식 기능으로 대체 가능한지 본다

04 Bedrock 은 Analytics 가 없다 → OTel 을 정책으로

05 modelPricing 으로 계약 단가 표시 , 정산은 AWS 와 대사

06 Gateway: SSO· 그룹 정책 ·spend limit, 사설망 전용

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

Thank you!

다음 : Capstone 쇼케이스 & Closing

Slack #claude-code-workshop · 질문은 언제든